

Chapter 9

Working with Databases and Tables



TEAM LING

Now that you know the big-picture view of how an RDBMS works, it's time to get into the details. This chapter focuses on the first component of SQL, the Data Definition Languages (DDL), by discussing the SQL commands to create (and delete) databases and tables, and view database, table, and field information.

A large part of this chapter is focused on a single command: the `CREATE TABLE` command. When designing a database, this is one of the most important commands you must know, because it enables you to decide the fundamental structure of your database. With the `CREATE TABLE` command, you can control, for example, how many fields each record must contain, which of those fields are optional, and what type of data can be entered into each field. MySQL lets you use different types of tables depending on your storage and data retrieval requirements. The `CREATE TABLE` command enables you to specify this information as well.

How to...

- Name and create a MySQL database
- Add tables to a database
- Decide the names and default values for the fields in a table
- Select the appropriate data type for a field
- Use enumerations to limit field input to a predefined list of values
- Specify which fields are optional and which are mandatory
- Select the table's primary and foreign key(s)
- Index frequently used fields for better performance
- Choose between MySQL's different table types (and select the one best suited for your needs)
- Alter a table definition after it's been created
- Back up and restore a table or a database
- View the structure and contents of a database or table
- Empty a table of its records
- Delete a database or table

Creating Databases

Because all tables are stored in a database, the first command you need to know is the `CREATE DATABASE` command, which initializes an empty database. Try it out by creating a database called `db2`:

```
mysql> CREATE DATABASE db2;  
Query OK, 1 row affected (0.05 sec)
```

Databases in MySQL are represented as directories on the disk, and tables are represented as files within those directories. Therefore, database names must comply with the operating system's (OS) restrictions on which characters are permissible within directory names. Database names cannot exceed 64 characters and names that contain special characters or consist entirely of digits or reserved words must be quoted with the backtick (```) operator.

TIP

Generally, it's considered good practice to start database names with an alphabetic character and to ensure they consist of only alphanumeric and underscore characters. Try to avoid using reserved MySQL keywords as database names.

9

You can select a particular database for use with the `USE` command. Select the `db2` database you just created to try this:

```
mysql> USE db2;  
Database changed
```

Once you select a database with the `USE` command, it becomes the default database for all operations.

Creating Tables

Because this is a new database, no tables are in it yet. To create a table, use the `CREATE TABLE` command, as in the following:

```
mysql> CREATE TABLE movies (  
-> mid int(10) UNSIGNED NOT NULL AUTO_INCREMENT,  
-> mtitle varchar(255) NOT NULL default '',  
-> myear year(4) NOT NULL default '0000',  
-> PRIMARY KEY (mid)  
-> ) TYPE=MyISAM;  
Query OK, 0 rows affected (0.10 sec)
```

The `CREATE TABLE` statement begins with the table name, followed by a set of parentheses. These parentheses enclose one or more *field definitions*, separated by commas. Each field definition contains the field name, its data type, and any special modifiers or constraints that apply. Following the closing parenthesis is an optional *table type specifier*, which tells MySQL which storage engine to use for this table.

Table and field names must conform to the same rules that apply to database names. MySQL tables are stored as files within the database directory and, as such, are subject to the host operating system's rules on file names.

TIP

To simplify moving your databases and tables between different operating systems, lowercase all your table names, and use underscores instead of spaces.

Specifying Field Data Types

When creating a MySQL table, specifying a data type for every field is necessary. This data type plays an important role in enforcing the integrity of the data in a MySQL database, and in making this data easier to use and manipulate. MySQL offers a number of different data types, which are summarized in Table 9-1.

The following sections examine each of these types in greater detail.

Numeric Types

For integer values, MySQL offers you a choice of the `TINYINT`, `SMALLINT`, `MEDIUMINT`, `INT`, and `BIGINT` types, which differ from each other only in the size of values they can store. Use the `TINYINT` and `SMALLINT` type for small integer values, the `INT` type for larger integer values, and the `BIGINT` type for extremely large values. For floating-point values, use the `FLOAT` and `DOUBLE` types for single-precision and double-precision floating point values, respectively. And, finally, for decimal values, use the `DECIMAL` data type.

When defining an integer field, you can include a width specifier in parentheses. This *width specifier* controls the padding MySQL applies to the field when retrieving it from the database. For a field defined as `BIGINT (20)`, MySQL will automatically pad the value to 20 characters before displaying it.

When defining floating-point and decimal fields, MySQL enables you to include both a width specifier and a *precision specifier*. For example, the declaration `FLOAT (7, 4)` specifies that displayed values will not contain more than seven digits, with four digits after the decimal point.

Type	Used For
TINYINT, SMALLINT, MEDIUMINT, INT, BIGINT	Integer values
FLOAT	Single-precision floating-point values
DOUBLE	Double-precision floating-point values
DECIMAL	Decimal values
CHAR	Fixed-length strings up to 255 characters
VARCHAR	Variable-length strings up to 255 characters
TINYBLOB, BLOB, MEDIUMBLOB, LONGBLOB	Large blocks of binary data
TINYTEXT, TEXT, MEDIUMTEXT, LONGTEXT	Longer blocks of text data
DATE	Date values
TIME	Time values or durations
YEAR	Year values
DATETIME	Combined date and time values
TIMESTAMP	Timestamps
ENUM	Fields that must contain one of a set of predefined mutually exclusive values
SET	Fields that can contain zero, one, or more of a set of predefined values

TABLE 9-1 MySQL Data Types

CAUTION

If you try to use a value that's too big for the field you're placing it in, MySQL will automatically truncate or round the value down to the maximum allowed value for that field.

Did you
know?

Zero-ing In

You can add two optional attributes to numeric type definitions: the `ZEROFILL` attribute, which pads a value with leading zeroes, and the `UNSIGNED` attribute, which forces a field to only accept positive values.

Character and String Types

MySQL lets you store strings up to 255 characters in length as either CHAR or VARCHAR types. The difference between these two types is simple: CHAR fields are fixed to the length specified at the time of definition, while VARCHAR fields can grow and shrink dynamically, based on the data entered into them. This makes VARCHAR fields more suitable for fields that accept variable-length data, and CHAR fields better for fields that always contain values of the same length.

Both CHAR and VARCHAR type definitions must include a width specifier in parentheses, as with numeric type definitions. Thus, the definition CHAR (10) creates a field whose length remains *exactly 10 characters* regardless of what is entered into it, while the definition VARCHAR (10) creates a field whose length can range *anywhere between 0 and 10 characters* depending on what is entered into it.

Text and Binary Types

MySQL enables you to store strings greater than 255 characters in length as either TEXT or BLOB types. The difference between TEXT and BLOB types is minimal at best: TEXT types are compared in a case-insensitive manner, while BLOB types are compared in a case-sensitive manner. For this reason, BLOBs are usually used to store binary data, while TEXT fields are used to store ASCII data.

Depending on the size of the string you're trying to store, MySQL offers you a choice of the TINYTEXT, TEXT, MEDIUMTEXT, and LONGTEXT types (for ASCII text blocks) and the TINYBLOB, BLOB, MEDIUMBLOB, and LONGBLOB types (for binary data).

Date and Time Types

For simple date and time values, MySQL offers the intelligently named DATE and TIME data types. The DATE type is used to store date values consisting of year, month, and day components, while the TIME type is used for time values or durations consisting of hour, minute, and second components. Both DATE and TIME types can be used for values in either numeric (YYYYMMDD and HHMMSS) or string ('YYYY-MM-DD' and 'HH:MM:SS') format.

If what you need is a combination of the two, consider using the DATETIME or TIMESTAMP types, both of which let you specify both date and time values in a single field. The difference between the two lies in how the values are stored: DATETIME fields are stored in the form 'YYYY-MM-DD HH:MM:SS', and TIMESTAMP fields are stored in the form YYYYMMDDHHMMSS.

Did you
know?

Time Out!

When inserting records into a table containing a `TIMESTAMP` field, MySQL automatically fills that field with the current date and time (assuming no other value was specified, of course). To accomplish the same thing with `DATETIME` fields, manually specify a value for the field with the `NOW()` function, which returns the current date and time.

Finally, for simple applications that only need to store the year, MySQL offers the special `YEAR` type, which accepts a 4-digit year value. It's worthwhile to use this value if your application deals mostly with the year component of a date value, because a field marked as `YEAR` occupies 1 byte on disk (as compared to a `DATETIME` or `DATE` field, which can occupy up to 8 bytes). MySQL is completely Y2K-compliant, and `YEAR` fields can accept any value in the range 1901 to 2155.

9

Enumerations

For situations where a field value must be selected from a predefined list of values, MySQL offers the `ENUM` and `SET` data types. For both these types, a list of predefined values must be included as part of the type definition. An `ENUM` field definition can contain up to 65,536 elements, while a `SET` field definition can hold up to 64 elements.

For a field marked as an `ENUM` field, only one of the predefined values may be selected, whereas for a field marked as a `SET` field, zero, one, or more than one of the predefined values may be selected. So, `ENUM` fields are best suited for mutually exclusive values, while `SET` fields are best suited for independent values. As an example, the definition `ENUM ('red', 'green', 'yellow')` forces entry of any one of the three values, while the definition `SET ('sugar', 'salt', 'pepper', 'mustard')` allows entry of none, one, or all of the four values.

CAUTION

With both `ENUM` and `SET` types, attempting to insert a value that does not exist in the predefined list of values will cause MySQL to insert either an empty string or a zero.

Selecting the Most Appropriate Data Type

Choosing the data type best suited to the values you expect to enter into the corresponding field is extremely important. When making this decision, take into account the following factors:

- The range and type of values that the field will hold
- The types of calculations you expect to perform on those values
- The manner in which the data is to be formatted for display purposes
- The manner in which the data is to be sorted and compared against other fields
- The available subtypes for each field and their storage efficiencies

By taking all these factors into consideration at the time of designing your database, you reduce the chance of incompatibilities and storage inefficiencies later.

Using the wrong data type can affect both the performance of your RDBMS and the types of operations you can perform on that field. For example, using a VARCHAR type on a field that is meant for numeric or date values could result in unexpected behavior when you perform calculations on it, just as using a large TEXT field for small string values could lead to a waste of space and inefficient indexing.

Adding Field Modifiers and Keys

You can apply a number of additional constraints, or modifiers, to a field to increase the consistency of the data that will be entered into it, and to mark it as “special” in some way. These modifiers can either appear as part of the field definition if they apply only to that specific field (for example, a default value for a field) or after all the field definitions if they relate to multiple fields (for example, a multicolumn primary key).

- You can specify whether the field is allowed to be empty or if it must necessarily be filled with data by placing the `NULL` and `NOT NULL` modifiers after each field definition.
- You can specify a default value for a field with the `DEFAULT` modifier. This default value is used if no value is specified for that field when inserting a record. In the absence of a `DEFAULT` modifier for `NOT NULL` fields, MySQL automatically inserts a nonthreatening default value into the field.

- You can have MySQL automatically generate a number for a field (by incrementing the previous value by 1) with the `AUTO_INCREMENT` modifier. This is particularly useful when you need to generate row numbers for each record in the table. However, the `AUTO_INCREMENT` modifier can only be applied to numeric fields that are both `NOT NULL` and belong to the `PRIMARY KEY`. A table may only contain one `AUTO_INCREMENT` field.
- For fields that accept string values, you can specify the character set for these values with the `CHARACTER SET` modifier (new in MySQL 4.1.1). However, this feature is only supported in MySQL's MyISAM, MERGE, and InnoDB table types (see the section entitled “Selecting a Table Type” for more on MySQL's table types).
- You can *index* a field with the `INDEX` modifier. When a field is indexed in this manner, MySQL no longer needs to scan each row of the table for a match when performing queries; instead, it can simply look up the index. This speeds up searches and reduces query response time. Indexing is recommended for fields that frequently appear in the `WHERE`, `ORDER BY`, and `GROUP BY` clauses of `SELECT` queries, and for fields used to join tables together.
- You can specify that values entered into a field must be either unique—that is, not duplicated—or `NULL` with the `UNIQUE` modifier.

NOTE

The `UNIQUE` modifier is actually a special type of index.

How to ...**Create a Primary Key Consisting of Multiple Fields**

In some tables, it is the combination of two or more fields, rather than a single field, that uniquely identifies a row. This is called a composite primary key; to create it, simply use a comma-separated field list instead of a single field in the `PRIMARY KEY` modifier.

- You can specify a primary key for the table with the `PRIMARY KEY` modifier. The `PRIMARY KEY` constraint can best be thought of as a combination of the `NOT NULL` and `UNIQUE` constraints because it requires values in the specified field to be neither `NULL` nor repeated in any other row. It thus serves as a unique identifier for each record in the table, and it should be selected only after careful thought has been given to the inter-relationships between tables.
- You can specify a foreign key for a table with the `FOREIGN KEY` modifier. The `FOREIGN KEY` modifier links a field in one table to a field (usually a primary key) in another table, setting up a base for relationships. However, foreign keys are only supported in MySQL's InnoDB table type; the `FOREIGN KEY` modifier is simply ignored in all other table types (see the section entitled "Selecting a Table Type" for more on MySQL's table types).

Selecting a Table Type

Following the field definitions and modifiers come one or more table modifiers, which specify table-level attributes. Of these, the most frequently used one is the `TYPE` modifier, which tells MySQL which table type to use. A number of such types are available, each with different advantages. Here is a list:

- **MyISAM** The MyISAM format is optimized for speed and reliability, it supports tables in excess of 4GB in size, and it can be compressed to save space. This is MySQL's default table type and, as such, contains numerous MySQL-specific optimizations and features.

Select this table type by adding `TYPE = MYISAM` to your `CREATE TABLE` statement.

Did you
know?

What's in a Name?

In MySQL, the terms *key* and *index* are synonymous. `PRIMARY KEY` fields are automatically indexed by MySQL, while `FOREIGN KEY` fields must be explicitly indexed by the user.

- **InnoDB** The successor to the MyISAM format, the InnoDB format, is the most sophisticated table type available in MySQL. It supports transactions and foreign keys (the only MySQL table type to do both), and allows multiple simultaneous users to execute `SELECT` statements; this improves performance and query response times. InnoDB tables are fully portable between different operating systems, and include crash recovery features to avoid data corruption or loss.

Select this table type by adding `TYPE = INNODB` to your `CREATE TABLE` statement.

- **HEAP** A HEAP table is stored in memory, making it extremely fast. This format is optimized for temporary tables and it is rarely used for other purposes. This is because the data in a HEAP table is available only while the server is running, and is automatically erased when the server shuts down and the memory is flushed.

Select this table type by adding `TYPE = HEAP` to your `CREATE TABLE` statement.

- **BerkeleyDB** The BerkeleyDB format is one of the more advanced table formats supported by MySQL. It supports transactions, checkpoints, crash recovery, and page-level locking. However, it also has certain disadvantages: BerkeleyDB tables are not easily portable between different operating systems and they lack many of the optimizations of the MyISAM format, making them slower and less memory efficient.

Select this table type by adding `TYPE = BDB` to your `CREATE TABLE` statement.

- **MERGE** The MERGE table format makes it possible for a collection of MyISAM tables to be treated as one, by combining them into a single “virtual” table. This table format makes improving performance or increasing query efficiency possible in certain situations; however, it can only be used for tables that are completely identical in their internal structure.

Select this table type by adding `TYPE = MERGE` to your `CREATE TABLE` statement.

- **ISAM** The forerunner of the newer MyISAM format, the ISAM format is primarily offered for compatibility with older MySQL tables. It lacks many of the features of the MyISAM format, cannot handle large tables, and is more prone to fragmentation (which degrades performance).

Select this table type by adding `TYPE = ISAM` to your `CREATE TABLE` statement.

Most of the time, you won't need to look further than the default MyISAM table type. Use the InnoDB table type only if you want to use advanced features like transactions and foreign keys, use the MERGE table type if you need to query multiple similar tables simultaneously, and use the HEAP table type if you need a temporary data storage area.

Now that you know what the various components of a `CREATE TABLE` statement are, try it out for yourself by revisiting the tables in Chapter 8 and writing the corresponding `CREATE TABLE` statement for each. Here's how:

```
mysql> CREATE TABLE persons (
-> pid int(11) NOT NULL auto_increment,
-> pname varchar(255) NOT NULL default '',
-> PRIMARY KEY (pid)
-> ) TYPE=MyISAM;
Query OK, 0 rows affected (0.09 sec)
mysql> CREATE TABLE roles (
-> mid int(11) NOT NULL default '0',
-> pid int(11) NOT NULL default '0',
-> role enum('A','D') NOT NULL default 'A',
-> PRIMARY KEY mid (mid,pid,role)
-> ) TYPE=MyISAM;
Query OK, 0 rows affected (0.11 sec)
```

The `CREATE TABLE` statement can be one of the most complex statements in the SQL lexicon. Because you'll use it frequently when designing a database, you should spend some time reading about it in detail in the MySQL manual, at http://dev.mysql.com/doc/mysql/en/CREATE_TABLE.html.

Did you
know?

It Was Here Just a Second Ago...

MySQL also lets you create temporary tables in memory with the `CREATE TEMPORARY TABLE` command. These tables remain in existence only for the duration of a single MySQL session and are automatically deleted when the client that created them closes its connection with the MySQL server. Temporary tables come in handy for transient, session-based data or calculations. And, because they're session-dependant, two different sessions can use the same table name without conflicting.

Altering Tables

Table definitions created with the `CREATE TABLE` command are not set in stone—you're free to alter them at a later date as well. The SQL command to do this is the `ALTER TABLE` command. It lets you add or delete fields; alter field types; add, remove, or modify keys; alter the table type; and change the table name (among other things). The following sections discuss these capabilities in greater detail.

Altering Table and Field Names

To alter a table name, use an `ALTER TABLE` command with a supplementary `RENAME` clause. The following example demonstrates, by renaming table `bills` to `invoices`:

```
mysql> ALTER TABLE bills RENAME TO invoices;
Query OK, 0 rows affected (0.11 sec)
```

An alternative is to use the `RENAME TABLE` command, which does the same thing:

```
mysql> RENAME TABLE bills TO invoices;
Query OK, 0 rows affected (0.06 sec)
```

You can just as easily alter a field name. Here's an example, which uses the `ALTER TABLE` command with a `CHANGE` clause to modify the name of field `address` to `address1`:

```
mysql> ALTER TABLE users CHANGE address address1 VARCHAR(255);
Query OK, 0 rows affected (0.17 sec)
```

Notice that you must include the column definition when changing a field name in this manner, or else MySQL will generate an error and disallow the operation.

Altering Field Properties

You can use the `CHANGE` clause discussed in the previous section to alter a field's type and properties as well, simply by using a new column definition instead of the

original one. Here's an example, which changes the field named `tel` defined as `VARCHAR(30)` to a field named `age` with definition `TINYINT(2)`:

```
mysql> ALTER TABLE users CHANGE tel age TINYINT(2);  
Query OK, 0 rows affected (0.05 sec)
```

When you `CHANGE` a field from one type to another, MySQL will automatically attempt to convert the data in that field to the new type. If the data in the field is inconsistent with the new field definition—for example, a field defined as `NOT NULL` contains `NULL` values, or a field marked as `UNIQUE` contains duplicate values—MySQL will generate an error. You can alter this default behavior by adding an `IGNORE` clause to the `ALTER TABLE` command that tells MySQL to ignore such inconsistencies.

Adding and Removing Fields and Keys

You can add a new field to a table by including an `ADD` clause in your `ALTER TABLE` command. The following example demonstrates, by adding a field named `salary` to the `employees` table:

```
mysql> ALTER TABLE employees ADD salary INT(7) NOT NULL;  
Query OK, 0 rows affected (0.06 sec)
```

You can also do the reverse—delete an existing field from a table—by using a `DROP` clause instead of an `ADD` clause. The following example removes the field added in the previous operation (together with any data it might have contained):

```
mysql> ALTER TABLE employees DROP salary;  
Query OK, 0 rows affected (0.05 sec)
```

You can delete a table's primary key with the `DROP PRIMARY KEY` clause, as illustrated here:

```
mysql> ALTER TABLE users DROP PRIMARY KEY;  
Query OK, 0 rows affected (0.06 sec)
```

and add a new primary key with the `ADD PRIMARY KEY` clause, as illustrated here:

```
mysql> ALTER TABLE users ADD PRIMARY KEY (id);  
Query OK, 0 rows affected (0.05 sec)
```

TIP

A table's primary key must always be NOT NULL.

Altering Table Types

You can alter the table type by adding a `TYPE` clause to the `ALTER TABLE` command, as in the following example:

```
mysql> ALTER TABLE data TYPE = INNODB;  
Query OK, 6 rows affected (0.11 sec)
```

Backing Up and Restoring Databases and Tables

The MySQL distribution comes with a powerful tool for backing up and restoring databases and tables: the `mysqldump` utility. When run on a database (or table), this command-line utility creates a text file containing all the SQL commands needed to re-create that database (or table) from scratch. The text file created by `mysqldump` can contain table definitions, table contents, or both, and it comes in handy both to create backups of your MySQL databases and to copy tables from one database or platform to another.

9

Backing Up Databases and Tables

To use the `mysqldump` utility to back up a database, invoke it from your command prompt and pass it a valid username, password, and database name. The following example illustrates, by using `mysqldump` on the `db2` database:

```
$ /usr/local/mysql/bin/mysqldump -u root -p hidden db2
```

Did you
know?

The Need for Speed

When you issue an `ALTER TABLE` command, MySQL first creates a copy of the original table, changes it, and then deletes the original table and replaces it with the changed copy. For this reason, `ALTER TABLE` operations on large tables may take a fair amount of time.

When `mysqldump` is invoked in this manner, it will connect to the MySQL server and retrieve the contents of the named database. The output of `mysqldump` is a series of SQL statements, which can be used to re-create the contents of the database. Because `mysqldump` sends everything to the screen by default, redirect the output to a file with the standard `>` redirection operator:

```
$ /usr/local/mysql/bin/mysqldump -u root -J  
-p hidden db2 > db2backup.sql
```

To back up only a specific table from the database, add the table name after the database name in the `mysqldump` command line. The following command backs up only the `movies` table from the `db2` database:

```
$ /usr/local/mysql/bin/mysqldump -u root -J  
-p hidden db2 movies > movies.sql
```

By default, `mysqldump` backs up both the table definitions and their contents. To only back up table definitions—useful if, for example, you’re attempting to re-create the empty tables in another database or platform—append the `--no-data` option to the `mysqldump` command line, as follows:

```
$ /usr/local/mysql/bin/mysqldump -u root -p hidden -J  
--no-data db2 movies > movies.sql
```

With the `--no-data` option, `mysqldump` stores the `CREATE TABLE` statement for the table, but none of its records.

Restoring Databases and Tables from Backup

Restoring a database or table from the text file created by `mysqldump` is extremely simple. All you need to do is pipe the file to the `mysql` command-line client, so that the SQL commands inside it are read and executed by the server. For example, to re-create the table previously backed up to the file `movies.sql` in the database `newdb`, use the following command:

```
$ mysql -u root -p secret -D newdb < movies.sql
```


Did you
know?

More Than One Way...

In MySQL 4.1, you can create an empty copy of a table by using the new `LIKE` clause in a `CREATE TABLE` command. Here is an example, which creates a new, empty copy of the `grades` table and the names it marks:

```
mysql> CREATE TABLE grades LIKE marks;  
Query OK, 0 rows affected (0.01 sec)
```

Notice that the database into which the table is restored must already exist and must be named on the `mysql` command line with the `-D` option. If you now inspect the `newdb` database with the `SHOW TABLES` command (as discussed in the section “Viewing Database, Table, and Field Information”), you will see the table in it.

9

Dropping Databases and Tables

To delete a database, use the `DROP DATABASE` command, which deletes the named database and all its tables permanently. Try this out by creating and dropping a database:

```
mysql> CREATE DATABASE music;  
Query OK, 1 row affected (0.05 sec)  
mysql> DROP DATABASE music;  
Query OK, 0 rows affected (0.49 sec)
```

Similarly, you can delete a table with the `DROP TABLE` command. Try this out by creating and dropping a table:

```
mysql> CREATE TABLE members ( memberId INT NOT NULL );  
Query OK, 0 rows affected (0.00 sec)  
mysql> DROP TABLE members;  
Query OK, 0 rows affected (0.00 sec)
```

The `DROP TABLE` command will immediately wipe out the specified table, together with all the data it contains—so use it with care!

If what you wanted was to empty the table of all records, use the `TRUNCATE TABLE` command instead, which internally `DROP`-s the table, and then re-creates it. The `AUTO_INCREMENT` counter, if one exists, is automatically reset in `TRUNCATE TABLE` operations (this does not happen if you simply delete all the records in the table with a `DELETE` command).

Here is an example:

```
mysql> TRUNCATE TABLE movies;
Query OK, 0 rows affected (0.01 sec)
```

Viewing Database, Table, and Field Information

You can view all available databases with the `SHOW DATABASES` command:

```
mysql> SHOW DATABASES;
+-----+
| Database |
+-----+
| db2      |
| mysql    |
| test     |
+-----+
3 rows in set (0.00 sec)
```

You can view available tables in a database with the `SHOW TABLES` command, as in the following:

```
mysql> SHOW TABLES FROM db2;
+-----+
| Tables_in_db2 |
+-----+
| movies        |
| persons       |
| roles         |
+-----+
3 rows in set (0.06 sec)
```

TIP

You can omit the FROM clause in the SHOW TABLES command if you've already selected the database with the USE command.

To see the structure of a particular table, use the DESCRIBE command, as in the following:

```
mysql> DESCRIBE movies;
```

Field	Type	Null	Key	Default	Extra
mid	int(10) unsigned		PRI	NULL	auto_increment
mtitle	varchar(255)				
myear	year(4)			0000	

```
3 rows in set (0.05 sec)
```

You can also

- Retrieve the SQL commands originally used to create the table with the SHOW CREATE TABLE command
- View a list of table indexes with the SHOW INDEX command
- Retrieve a list of table types supported by the server with the SHOW ENGINES command (in MySQL 4.1.2 and better)
- View a list of active connections to the server, as well as what each one is doing, with the SHOW PROCESSLIST command
- View a list of errors and warnings generated by the server with the SHOW ERRORS and SHOW WARNINGS commands (in MySQL 4.1.0 and better)
- Obtain server status (including information on server uptime, number of queries processed, and number of connections) with the SHOW STATUS command
- Obtain detailed information on the tables in a database (including information on the table type, the number of rows, the date and time of the last table update, and the lengths of indexes and rows) with the SHOW TABLE STATUS command, and
- View a list of available character sets (in MySQL 4.1.1 and better) with the SHOW CHARACTER SET command

The output of these commands cannot be reproduced here due to page-width considerations. Try them out for yourself to see how they work.

Summary

You should now have a clear understanding of how to work with MySQL's databases and tables. This chapter discussed most of the important aspects of the DDL, including how to create and delete databases, define table structures, work with field data types and modifiers, choose a MySQL table type, and use primary and foreign keys. It also showed you how to alter a table after it's been created, and how to back up and restore an existing table structure and/or its contents.

With your database(s) and table(s) defined, the next step is to begin adding data to them. That's what the next chapter is all about, but before you begin reading, take some time to learn more about the topics covered in this chapter by visiting the following links:

- A complete list of options to the `CREATE TABLE` statement, at [**http://dev.mysql.com/doc/mysql/en/CREATE_TABLE.html**](http://dev.mysql.com/doc/mysql/en/CREATE_TABLE.html)
- Detailed descriptions of MySQL's numerous data types, at [**http://dev.mysql.com/doc/mysql/en/Column_types.html**](http://dev.mysql.com/doc/mysql/en/Column_types.html)
- Detailed descriptions of MySQL's table types and guidelines on how to choose among them, at [**http://dev.mysql.com/doc/mysql/en/Table_types.html**](http://dev.mysql.com/doc/mysql/en/Table_types.html)
- More information on altering tables, at [**http://dev.mysql.com/doc/mysql/en/ALTER_TABLE.html**](http://dev.mysql.com/doc/mysql/en/ALTER_TABLE.html)
- More information on the `mysqldump` utility, at [**http://dev.mysql.com/doc/mysql/en/mysqldump.html**](http://dev.mysql.com/doc/mysql/en/mysqldump.html)
- A complete list of `SHOW . . .` commands, with output samples, at [**http://dev.mysql.com/doc/mysql/en/SHOW.html**](http://dev.mysql.com/doc/mysql/en/SHOW.html)